

SIGNALS REPORT

How I removed the salt and pepper noise:

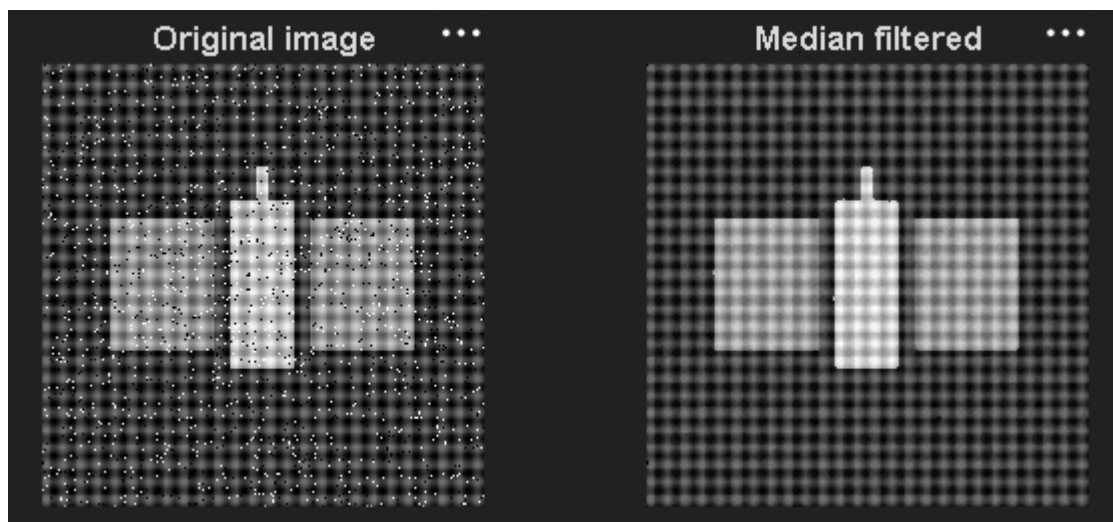
A **median filter** of size **3 x 3** slides across the image, taking the median of all pixels that fall into the matrix and creating the new image.

WHY A MEDIAN FILTER?

Since the image has salt and pepper noise, there are either **pure black** or **pure white** pixel at random places but are not that dense. The median filter slides across the image, arranges pixels in the matrix by pixel values and keeps the middle element in the filtered image.

Since pure white and pure black will lie either at the **starting** or the **end** of the arrangement, they will never be the median and hence they get removed!

I **didn't use an averaging filter** because it would get rid of sharp features and blur the image and we would see bright areas wherever there was salt and dark areas where there were pepper because of averaging. But the median filter doesn't affect any important features in the image.



How I used the fourier transform to suppress grid noise:

Plotting the FFT spectrum of all median-filtered images revealed a distinct pattern: The horizontal, periodic lines of the grid lead to a sharp bright vertical line in the FFT spectrum, passing through the center and the vertical lines of the grid lead to a sharp bright horizontal line in the FFT which also passes through the center. This results in a **plus symbol** right at the center of the FFT spectrum.

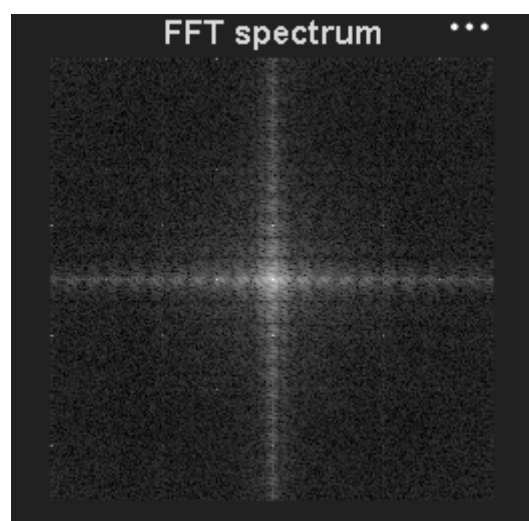
Why the plus pattern occurred?

In a 2D Fourier Transform, frequencies represent the **rate of change** in pixel intensities.

Vertical Grid Lines: If you trace a vertical line from top to bottom, the pixel color doesn't change (the rate of change in the y-direction is zero). However, if you move horizontally across the image, you hit the vertical lines over and over, creating a very high rate of change in the x-direction. Because the change is purely horizontal, the FFT maps all of this energy onto the horizontal axis passing through the center.

Horizontal Grid Lines: Moving horizontally along a horizontal line yields zero change, but moving vertically across them yields rapid, repeating changes. Therefore, all the energy from horizontal lines is mapped purely onto the vertical axis.

The noisy grids in the images results in an FFT spectrum that looks like this:



(for [cursed_schematic_02](#))

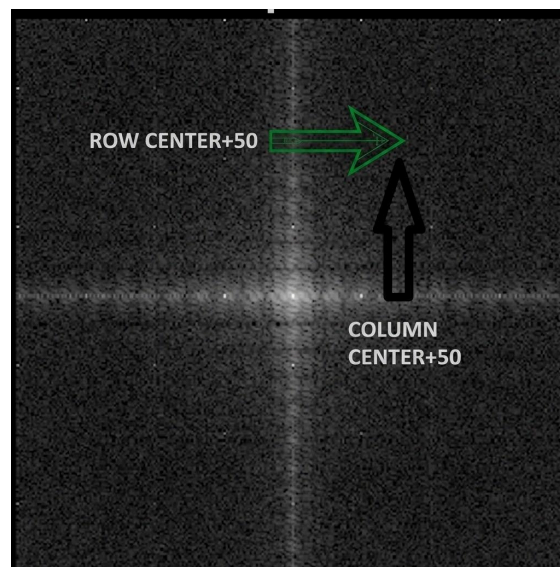
Creating the frequency mask and getting rid of the plus:

To suppress the grid noise without destroying the underlying schematic, I designed an adaptive frequency mask to target the distinct plus shaped (+) artifact in the magnitude spectrum. Rather than hardcoding the dimensions of this plus, the algorithm dynamically calculates its bounds based on the specific energy signature of the image.

1) Choosing the Threshold:

Because the thickness and intensity of the grid lines can vary between different images, hardcoding the mask's width would make the filter brittle. Instead, the algorithm determines the necessary width dynamically:

- **Checkpoints:** The script samples a specific checkpoint row and column located at a safe distance from the center (center + 50 pixels).



- **Peak Detection:** It identifies the maximum intensity (the brightest noise pixel) along these checkpoints. (See [find_threshold](#) function)
- **Thresholding:** The script then traverses outward along the array from this peak. It calculates the boundary of the + shape by finding the exact index where the pixel intensity drops below **70% of the peak noise magnitude** (a [0.7](#) sensitivity multiplier). This index perfectly captures the width of the noise band along both the horizontal and vertical axes. (See [find_threshold](#) function).

(I determined the sensitivity multiplier by plotting the pixel values for the row checkpoint and the column checkpoint)

2. Constructing the Mask

With the exact bounds of the grid noise isolated (`plus_width_along_h` and `plus_width_along_v`), the spatial mask is constructed:

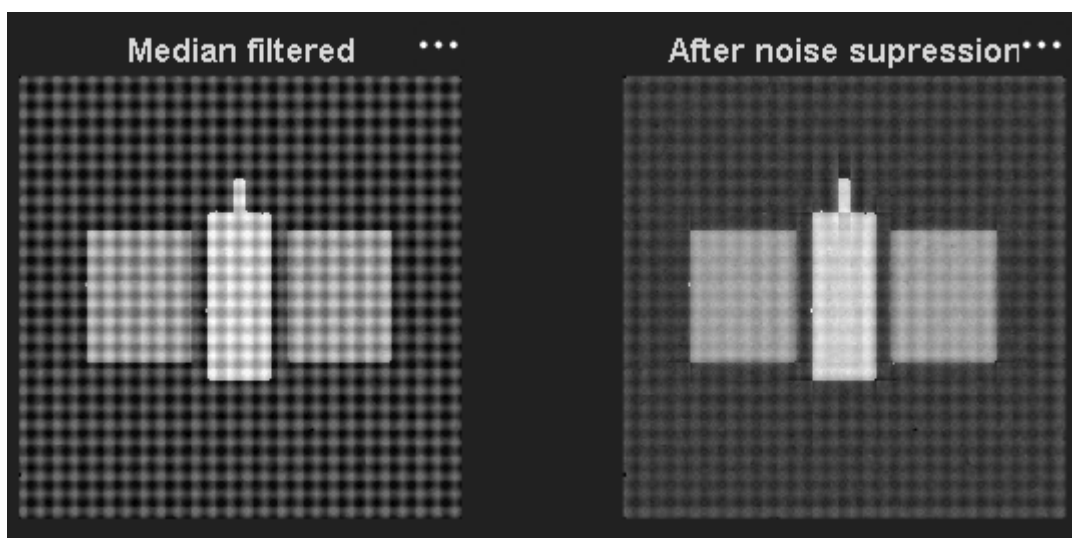
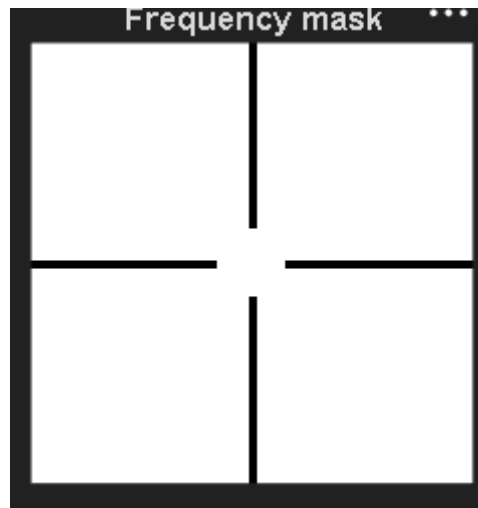
- The mask initializes as a purely white image (a matrix of `1`s), which allows all frequencies to pass through unchanged by default.
- In the regions defined by our dynamic widths, the mask values are reduced to a `darkening_factor` of `0.3`.
- By attenuating the noise bands by 70% (`0.3`) rather than aggressively zeroing them out (`0`), the filter acts as a soft notch. This significantly reduces the grid visibility while preserving the important features of the image that might lie within the threshold.

3. Preserving the DC Component:

A critical feature of the mask is the implementation of a **DC Threshold** (`DC_threshold = 20`).

The horizontal and vertical noise bands intersect precisely at the center of the spectrum. This exact center pixel is the **DC component**, which represents the frequency of zero and dictates the overall average brightness of the entire spatial image.

If the mask blindly attenuated the entire `+` shape, it would also crush this DC component, causing the final reconstructed image to turn completely black or extremely dark. To prevent this, the mask logic explicitly skips a 20 pixel radius around the center. This hole in the mask allows the image's base brightness and core structural contrast to remain entirely intact while the high-frequency grid noise extending outward is suppressed!



Applying morphological operations:

The suppression of the grid noise leaves behind fragmented remnants: scattered "tiny black pixels" (false negatives within bright regions) and "tiny white pixels" (false positives within dark regions).

I applied the morphological operations (opening and closing straight from the matlab image processing onramp) to get rid of these.

I followed a three stage pipeline: **Closing > Opening > Closing**. All of them use a disk shaped structuring element.

1) Structuring Element (SE) Sizing:

Rather than relying on hardcoded parameters, the radius of the disk-shaped structuring element is dynamically determined by the spatial frequency of the grid (`grid_frequency`). A higher grid frequency implies narrower grid lines, necessitating a smaller SE, while a lower frequency requires a larger SE because the size of the SE should be greater than the size of the noise it has to remove.

2) The three stage pipeline:

Stage 1: Initial Closing (`imclose`):

- Operation: Dilation followed by Erosion
- SE Size: `pixel` (Determined by grid frequency).
- Purpose: The primary objective of this initial step is to eliminate the dark artifacts (tiny black pixels) that fracture the background and the main white object.. It expands the bright regions, effectively engulfing the small black holes left by the grid.

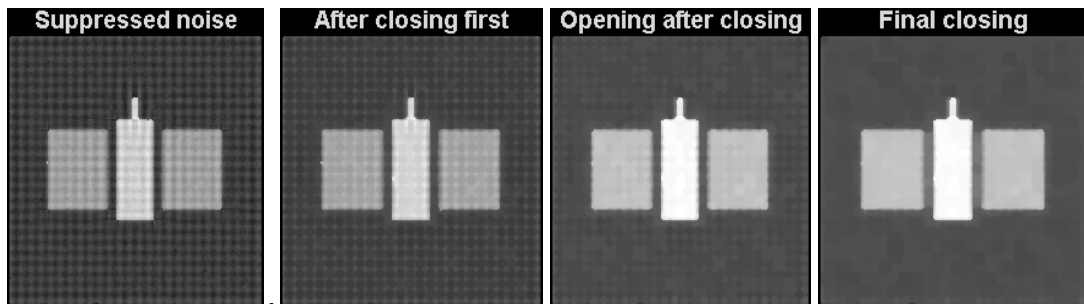
Stage 2: Intermediate Opening (`imopen`):

- Operation: Erosion followed by Dilation.
- SE Size: `min(4, pixel)`.
- Purpose: After closing, the image may still contain residual bright artifacts (tiny white pixels) scattered in the background. Opening removes these small bright objects.
- Constraint: The SE radius here is strictly capped at a maximum of `4`. If a larger SE were used, the erosion phase of the opening operation could "eat" into the edges or thin protruding features of the primary white object, causing irreversible loss. The capped size guarantees that only the micro-artifacts are destroyed.

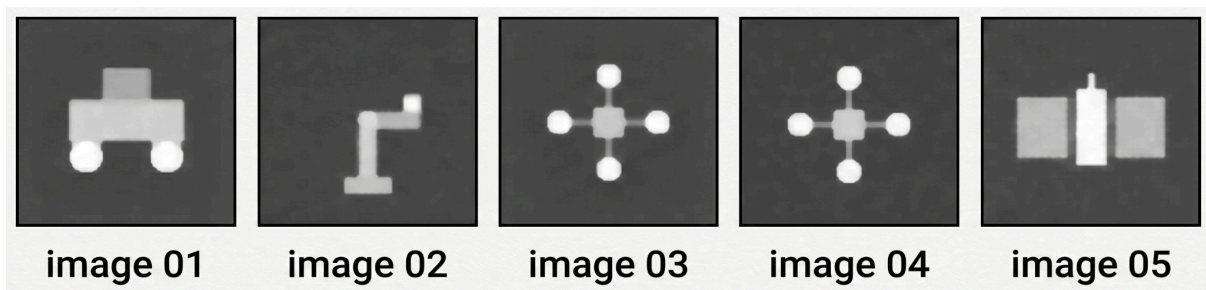
Stage 3: Final Closing (`imclose`):

- Operation: Dilation followed by Erosion.
- SE Size: `pixel + 1`.
- Purpose: The intermediate opening process, while removing white noise, still leaves some larger patches of black and white. A final closing with a larger radius ensures the larger black patches disappear and mix into the

background giving the final restored image. We could have done opening here to remove the larger white patches instead, but because the radius of the SE is bigger opening would eat away important white features



FINAL RESULT:



BONUS:

Blueprint creation:

The `generate_blueprint` function converts the cleaned grayscale image into an architectural blueprint by extracting structural boundaries and overlaying them onto a solid blue RGB canvas.

1) Feature Extraction: It utilizes the Canny edge detection algorithm with a strict sensitivity threshold (0.125) determined experimentally to isolate the dominant outlines of the main object.

2) Morphological Operations: To ensure edges are continuous and visible, the binary mask undergoes a two-step morphological process:

- **Dilation (7x7 square):** Expands boundaries to bridge gaps and connect disjointed lines.
- **Erosion (5x5 square):** Slightly thins the expanded lines so they remain connected but are visually constrained to an appropriate thickness.

3) Final Overlaying: The script initializes a background canvas with specific RGB values (R=0.1, G=0.3, B=0.6) to create a deep blue. The edge mask is then used as an index to overwrite the boundary pixels with an off-white color (0.6 across all channels). Finally, the three channels are concatenated to produce the complete RGB blueprint.

Example blueprint:

